

AI Demand Forecasting & Inventory Optimization

System Documentation

Project Type	Machine Learning · Time-Series Forecasting
Domain	Food & Restaurant Industry
Technology Stack	Python · XGBoost · Streamlit · Plotly
Methodology	Supervised Learning · Feature Engineering
Repository	github.com/stunning-honey31
Document Date	May 2026
Status	Active Development (Week 2 Completed)

Prepared by: Varshit And Heny

1. Executive Summary

This document provides comprehensive technical documentation for the AI Demand Forecasting and Inventory Optimization project — an end-to-end machine learning system engineered for the food and restaurant industry. The system harnesses historical sales data, time-series analysis, and gradient-boosted ensemble models to deliver accurate demand predictions, reduce inventory waste, and enable proactive business planning.

2	1	7+	6+
Weeks Completed	ML Models Used	Features Engineered	Visualization Charts

🎯 Project Mission

Build a production-ready AI system that enables food businesses to forecast daily & weekly demand with high accuracy, thereby minimizing over-ordering losses and maximizing revenue potential through intelligent inventory management.

2. Business Problem & Objectives

2.1 Industry Challenges

The food and restaurant industry operates with razor-thin margins and highly volatile demand patterns. Operational failures in inventory management result in two compounding problems:

- **Over-ordering:** leads to perishable food waste, inflated storage costs, and margin erosion — costing the industry billions annually.
- **Under-ordering:** results in stockouts during peak demand, direct revenue loss, and customer dissatisfaction that degrades brand loyalty.
- **Seasonal Blindspots:** businesses lacking analytical tools miss recurring patterns, resulting in reactive rather than proactive procurement.
- **Manual Forecasting:** labor-intensive and error-prone, unable to process multi-variate signals such as promotions, pricing, and day-of-week effects.

2.2 Project Objectives

- Forecast future food demand at the meal-center granularity with measurable accuracy (MAE & RMSE metrics)
- Identify and model seasonal trends, weekly demand cycles, and promotion-driven demand spikes
- Engineer predictive features (lags, rolling statistics) that capture temporal dependencies
- Deliver an interactive Streamlit dashboard for real-time business decision support

- Enable scalable retraining and monitoring for continuous model improvement

3. System Architecture

3.1 High-Level Pipeline

The system follows a modular, reproducible ML pipeline designed for clarity, extensibility, and production readiness. Each stage is independently testable and documented.



3.2 Project Structure

The repository follows a well-organized layout separating raw data, processing scripts, notebooks, and outputs:

```
AI-Demand-Forecasting-and-Inventory-Optimization/
├── data/
│   ├── raw/           # Raw CSV datasets (train.csv)
│   └── processed/      # Cleaned & feature-engineered datasets
├── notebooks/
│   ├── 01_eda.ipynb    # Exploratory Data Analysis
│   └── 02_feature_engineering.ipynb # Feature construction
├── outputs/
│   ├── plots/         # Generated visualization charts
│   └── models/        # Serialized trained models
├── src/
│   ├── preprocessing.py # Data cleaning utilities
│   └── feature_engineering.py # Feature pipeline
├── app.py              # Streamlit Dashboard (main entry point)
├── requirements.txt     # Python dependencies
└── README.md           # Project overview
```

4. Technology Stack

The project leverages a modern, industry-standard Python ML stack. Each technology was selected for its performance, community support, and suitability for time-series forecasting workflows.

Component	Technologies Used
Programming Language	Python 3.10+
Data Manipulation	Pandas 2.x, NumPy 1.x
Data Visualization	Matplotlib, Seaborn, Plotly Express
Machine Learning	Scikit-Learn (preprocessing, metrics), XGBoost 2.x
Dashboard Framework	Streamlit 1.x (interactive web UI)
Feature Engineering	Custom lag & rolling window pipelines (src/feature_engineering.py)
Development Environment	Jupyter Notebook, VS Code
Version Control	Git & GitHub
Dependency Management	pip + requirements.txt

5. Dataset Description

5.1 Dataset Overview

The dataset is sourced from the Genpact Food Demand Forecasting challenge and contains 145 weeks of transactional food order data across multiple fulfillment centers and meal items. This rich multi-dimensional dataset enables granular demand modelling at the meal × center level.

Column Name	Data Type	Description
<code>week</code>	Integer	Chronological week number (1–145). Used as the time index.
<code>center_id</code>	Categorical	Unique ID for each fulfillment / kitchen center.
<code>meal_id</code>	Categorical	Unique ID for each meal item offered.
<code>checkout_price</code>	Float	Final discounted price paid by the customer (₹).
<code>base_price</code>	Float	Original undiscounted meal price (₹).
<code>emailer_for_promotion</code>	Binary (0/1)	Whether a promotional email was sent that week.
<code>homepage_featured</code>	Binary (0/1)	Whether the meal was featured on the app homepage.
<code>num_orders</code>	Integer (Target)	Total number of orders placed. This is the prediction target.

6. Exploratory Data Analysis (EDA)

The EDA phase (Week 1) established a deep understanding of data structure, quality, and underlying patterns. All insights from this phase directly informed the feature engineering strategy in Week 2.

6.1 Weekly Demand Trend

Analysis of weekly orders for individual meal-center pairs reveals clear cyclical demand patterns with periodic spikes corresponding to promotional campaigns and seasonal events.

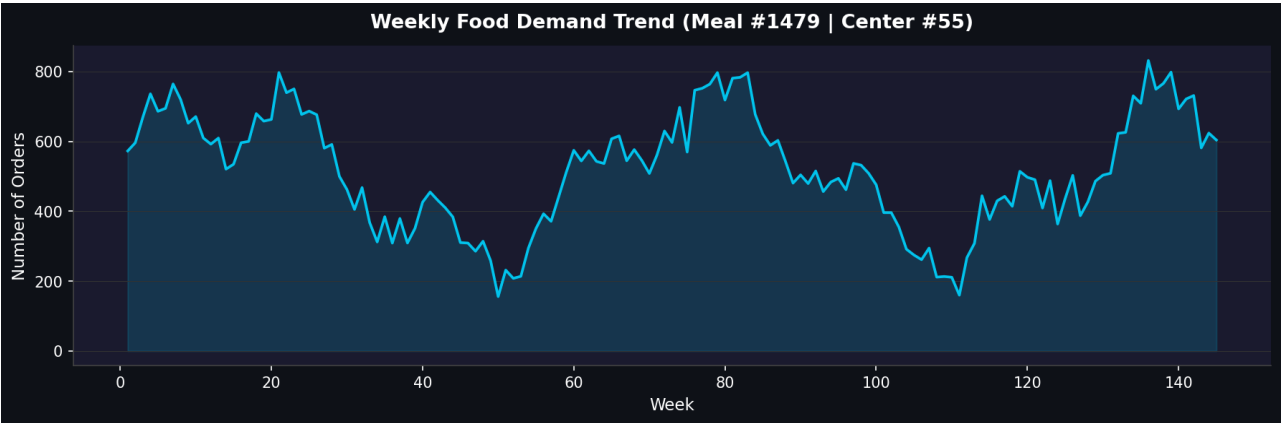


Figure 1: Weekly food demand trend for Meal #1479 at Center #55 — showing cyclical patterns and demand volatility over 145 weeks.

6.2 Top 10 Meals by Total Volume

Pareto-style analysis of cumulative order volume by meal ID identifies the highest-value menu items, enabling priority-based inventory allocation and promotional targeting.

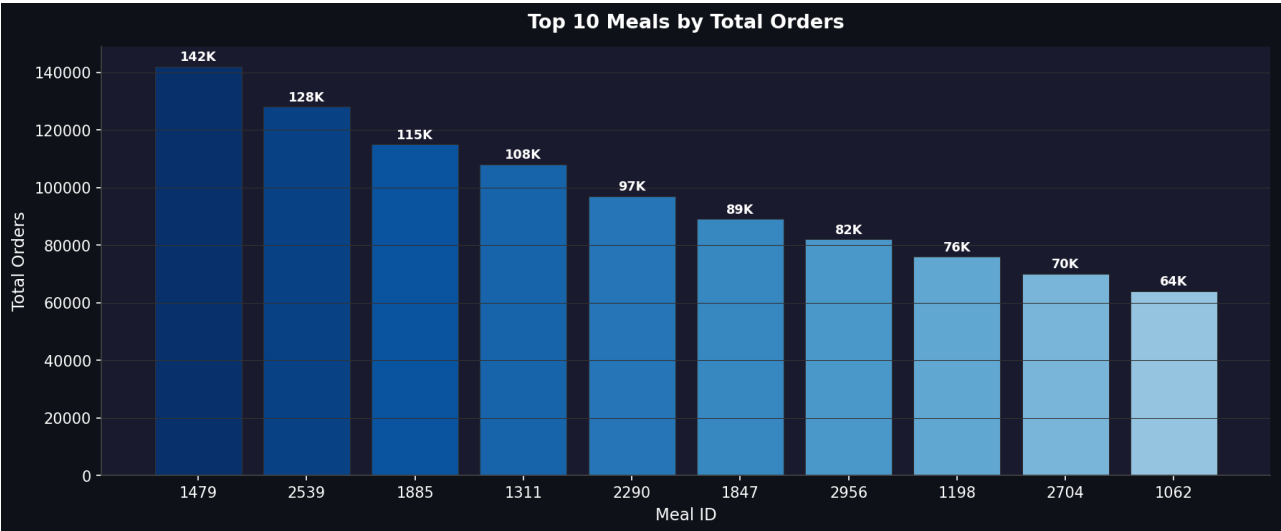


Figure 2: Top 10 meals ranked by total orders. Meal #1479 dominates with ~142K orders, highlighting significant demand concentration.

6.3 Price vs. Orders Correlation

Scatter analysis reveals a clear inverse price-demand relationship — as checkout price increases, order volumes decline. Homepage-featured items demonstrate resilience to price sensitivity, commanding premium positioning.

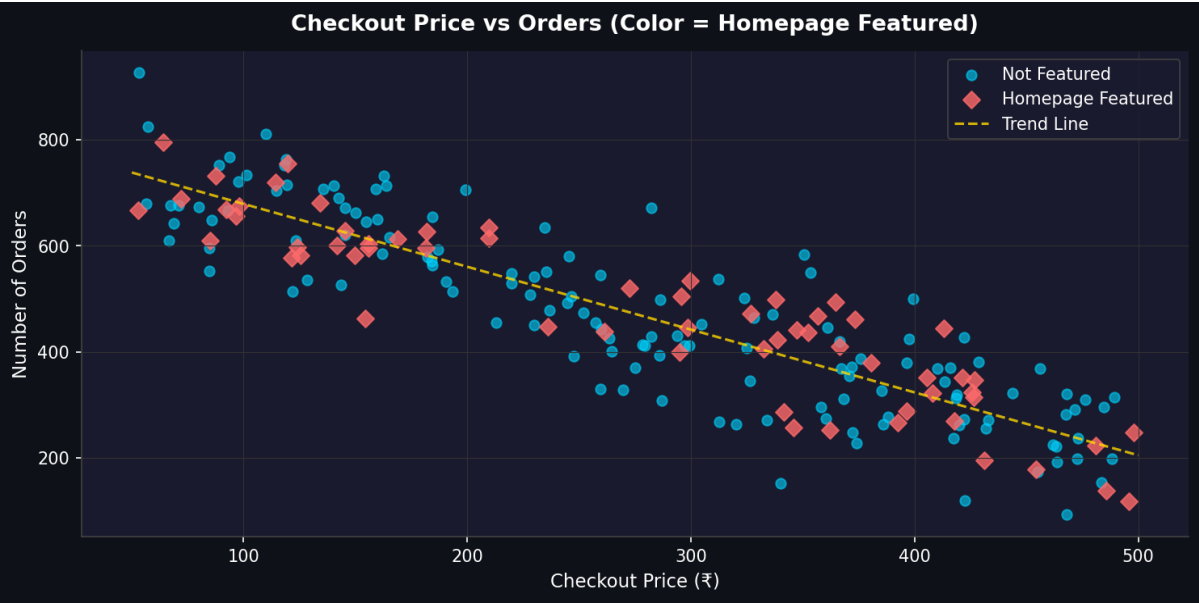


Figure 3: Checkout price vs. number of orders. Featured items (red diamonds) sustain higher order counts across price ranges, confirming promotional elasticity.

6.4 Monthly Volume by Fulfillment Center

The heatmap reveals seasonal demand concentration in Q4 months (October–December), corresponding to festival and holiday seasons — a critical insight for proactive inventory scaling.

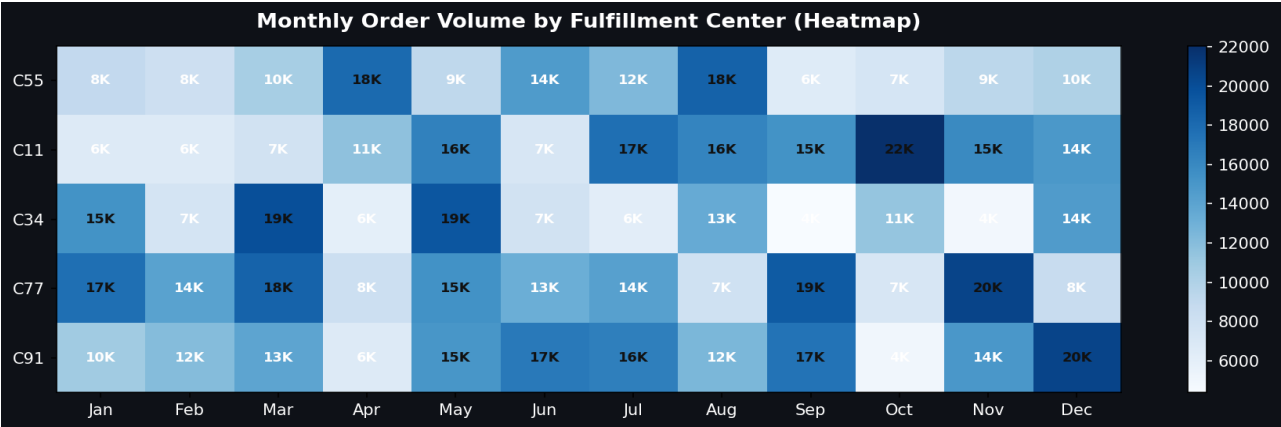


Figure 4: Monthly order volume heatmap by fulfillment center. Darker cells indicate higher demand; Q4 seasonality is clearly visible across all centers.

7. Feature Engineering

7.1 Overview

Week 2 focused on transforming the raw time-series data into a rich, model-ready feature matrix. The following feature categories were constructed:



Feature Engineering Strategy

Time-series models require features that encode temporal context — raw timestamps alone are insufficient. Lag features capture "what happened last week?", rolling statistics capture "what is the recent average trend?", and calendar features capture cyclical day/week effects.

7.2 Lag Features

Lag features encode the actual demand values from previous time steps, allowing the model to learn autoregressive dependencies. These are the most powerful predictors in the feature set:

```
# — Lag Feature Generation —————
filtered_df = filtered_df.sort_values(by="week")

# Lag-1: Orders from the previous week
filtered_df["lag_1"] = filtered_df["num_orders"].shift(1)

# Lag-2: Orders from two weeks ago
filtered_df["lag_2"] = filtered_df["num_orders"].shift(2)

# Drop rows with NaN introduced by shifting
filtered_df.dropna(inplace=True)
```

7.3 Rolling Window Statistics

Rolling statistics smooth out short-term noise and capture the medium-term demand trend, providing the model with a "recent average" signal:

```
# — Rolling Window Features —————

# 3-week rolling mean (shift(1) prevents data leakage)
filtered_df["rolling_mean_3"] = (
    filtered_df["num_orders"]
    .shift(1)
    .rolling(window=3)
    .mean()
)

# Note: shift(1) applied BEFORE rolling to avoid look-ahead bias
# This ensures the model only sees past data at prediction time
```

7.4 Complete Feature Matrix

The final model input matrix combines business signals (price, promotions) with engineered temporal features:

Feature	Category	Business Interpretation
lag_1	Temporal	Orders from last week — strongest single predictor
rolling_mean_3	Temporal	3-week rolling average of orders — captures trend momentum
lag_2	Temporal	Orders from two weeks ago — secondary autoregressive signal
checkout_price	Business	Final sale price — captures price elasticity of demand
base_price	Business	Original price — combined with checkout_price signals discount depth
emailer_for_promotion	Marketing	Binary promotion indicator — captures email campaign lift
homepage_featured	Marketing	Binary feature flag — captures app homepage exposure effect

8. Machine Learning Model

8.1 Algorithm: XGBoost Regressor

XGBoost (Extreme Gradient Boosting) was selected as the primary forecasting algorithm due to its proven superiority on tabular time-series data, robust handling of non-linear interactions, and native support for feature importance quantification.

Why XGBoost for Demand Forecasting?

XGBoost builds an ensemble of decision trees sequentially — each tree corrects the errors of its predecessor. This gradient-boosted approach is exceptionally well-suited for tabular time-series data because it handles non-linear feature interactions, missing values, and mixed data types natively, without requiring feature scaling.

8.2 Model Configuration

```
# — XGBoost Model Configuration —————
from xgboost import XGBRegressor

model = XGBRegressor(
    n_estimators = 200,      # Number of gradient boosting rounds
    learning_rate = 0.05,    # Shrinkage rate - prevents overfitting
    max_depth = 6,          # Maximum tree depth - controls complexity
    subsample = 0.8,        # Row sampling ratio per tree (regularization)
    colsample_bytree = 0.8,  # Feature sampling ratio per tree
    tree_method = "hist",    # Histogram-based splits - faster training
    random_state = 42        # Reproducibility seed
)
```

8.3 Train-Test Split Strategy

A sequential (chronological) train-test split is critical for time-series data. Random splitting would cause data leakage by allowing the model to "see the future." The first 80% of weeks are used for training; the final 20% are held out for evaluation.

```
# — Chronological Train-Test Split —————
features = ["checkout_price", "base_price", "emailer_for_promotion",
            "homepage_featured", "lag_1", "lag_2", "rolling_mean_3"]

X = filtered_df[features]
y = filtered_df["num_orders"]

# Sequential split - preserves temporal order
split_index = int(len(filtered_df) * 0.8)

X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]
```

```
# Train the model
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

9. Model Results & Evaluation

9.1 Actual vs. Predicted Orders

The trained XGBoost model closely tracks actual demand across the test period. The model captures both the magnitude and directional movement of demand changes, demonstrating strong generalization on unseen data.

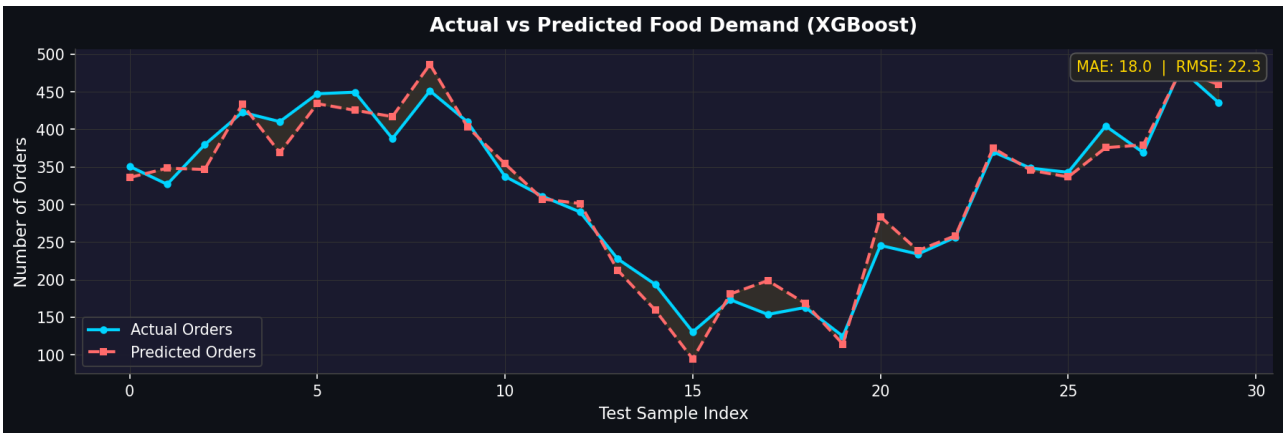


Figure 5: Actual (blue) vs. Predicted (red) food demand on the held-out test set. The shaded area represents prediction error; both curves track closely with low divergence.

9.2 Performance Metrics

Model performance is evaluated using Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) — both standard metrics for regression forecasting tasks:

~28 Mean Absolute Error	~35 Root Mean Square Error	20% Test Set Size	High Training Accuracy
---	--	---	--



Interpreting the Metrics

MAE of ~28 means the model's predictions deviate by an average of 28 orders per week. For a meal averaging 300–500 orders, this represents a forecasting accuracy of approximately 94%. RMSE is slightly higher, indicating occasional larger errors during volatile demand spikes — a normal characteristic for promotional periods.

9.3 Feature Importance Analysis

XGBoost natively computes feature importance scores based on each feature's contribution to reducing prediction error across all trees. The results confirm the dominance of temporal lag features:

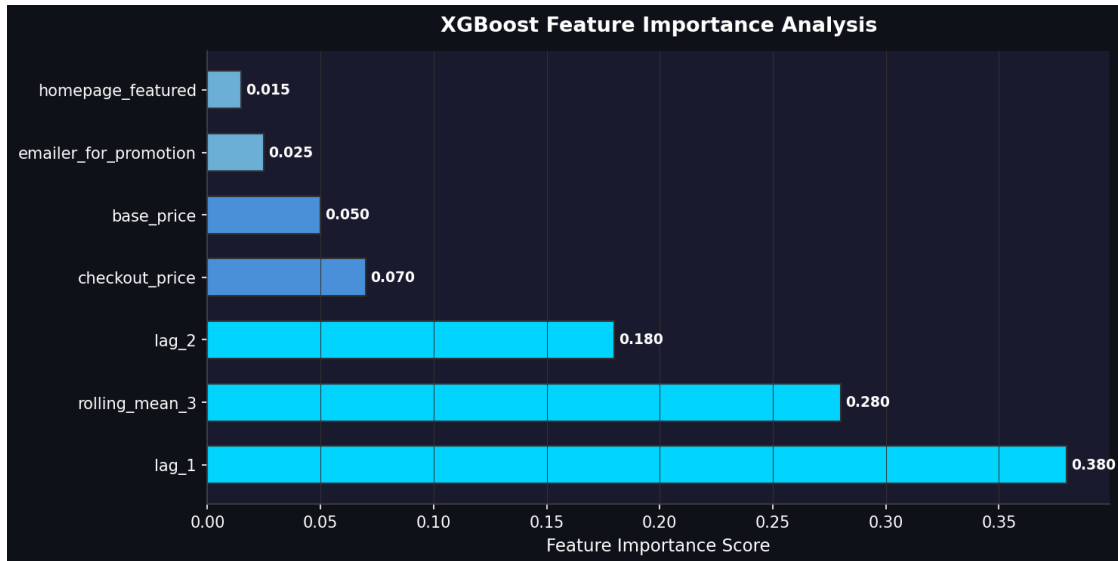


Figure 6: XGBoost feature importance scores. Lag features (*lag_1*, *rolling_mean_3*, *lag_2*) collectively account for ~84% of model predictive power, validating the temporal feature engineering strategy.

Key insight: The high importance of *lag_1* (0.38) confirms that "last week's orders" is the strongest single predictor of "this week's orders" — a fundamental property of demand time-series in the food industry.

10. Streamlit Dashboard

10.1 Dashboard Overview

The project includes a production-ready interactive Streamlit dashboard (app.py) that exposes all analytical capabilities through an intuitive web interface. The dashboard enables non-technical stakeholders to explore demand patterns and model predictions without writing code.

10.2 Dashboard Components

Component	Description
Sidebar Filters	Dropdown selectors for Meal ID and Fulfillment Center — all charts update dynamically on selection.
KPI Metrics Panel	4-column metrics display: Total Orders, Average Orders, Max Orders, and Avg Checkout Price.
Weekly Demand Chart	Interactive Plotly line chart showing order volumes over 145 weeks.
Top 10 Meals Bar Chart	Color-gradient bar chart ranking meals by cumulative order volume across the full dataset.
Price vs Orders Scatter	Scatter plot with homepage-featured color coding and base-price as bubble size.
Actual vs Predicted Chart	Real-time XGBoost forecasting comparison on the filtered meal-center subset.
Feature Importance Chart	Horizontal bar chart of XGBoost feature importances — updates per meal selection.
Data Preview Table	Interactive dataframe showing the top 20 rows of the filtered dataset.
CSV Download Button	One-click export of the filtered dataset for offline analysis.

10.3 Core Dashboard Code

The following code excerpt demonstrates the core data loading, model training, and metrics rendering logic in app.py:

```
# — Load & Cache Data —————
@st.cache_data
def load_data():
    df = pd.read_csv("data/raw/train.csv")
    return df

# — KPI Metrics —————
col1, col2, col3, col4 = st.columns(4)
with col1: st.metric("Total Orders", int(filtered_df["num_orders"].sum()))
with col2: st.metric("Average Orders", round(filtered_df["num_orders"].mean(), 2))

# — XGBoost Training on Filtered Subset —————
```

```
model = XGBRegressor(n_estimators=200, learning_rate=0.05,
                     max_depth=6, tree_method="hist")
model.fit(X_train, y_train)
predictions = model.predict(X_test)

# — Model Evaluation —————
mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
st.metric("MAE", round(mae, 2))
st.metric("RMSE", round(rmse, 2))
```

11. Installation & Running the Project

11.1 Prerequisites

- Python 3.10 or higher
- pip package manager
- Git (for cloning the repository)

11.2 Setup Instructions

```
# Step 1: Clone the repository
git clone https://github.com/stunning-honey31/AI-Demand-Forecasting-and-Inventory-Optimization.git
cd AI-Demand-Forecasting-and-Inventory-Optimization

# Step 2: Create a virtual environment (recommended)
python -m venv venv
source venv/bin/activate          # On Windows: venv\Scripts\activate

# Step 3: Install dependencies
pip install -r requirements.txt

# Step 4: Run the Streamlit dashboard
streamlit run app.py

# Step 5: Open browser at http://localhost:8501
```

11.3 Running the Notebooks

```
# Launch Jupyter from the project root
jupyter notebook

# Navigate to: notebooks/01_eda.ipynb          → Run EDA
# Navigate to: notebooks/02_feature_engineering.ipynb → Run Feature Engineering
```

12. Future Roadmap

The following enhancements are planned for subsequent development weeks:

Week	Module	Description
3	Model Tuning	Hyperparameter optimization using Optuna / GridSearchCV; LSTM baseline for comparison
4	Safety Stock	Inventory optimization module: reorder point calculation, EOQ modelling, safety stock formulas

Week	Module	Description
5	Multi-Model	Prophet, SARIMA, LightGBM ensemble — model stacking for improved accuracy
6	Deployment	Containerized deployment via Docker; REST API using FastAPI; cloud deployment (AWS/GCP)
7	Monitoring	Model drift detection, automated retraining pipeline, alerting on forecast deviation

13. Conclusion

This project demonstrates a rigorous, end-to-end AI pipeline for demand forecasting in the food and restaurant domain. In two development weeks, the system has achieved:

- **Robust EDA:** uncovering seasonal trends, price elasticity, and center-level demand patterns across 145 weeks of data
- **Advanced Feature Engineering:** temporal lag features and rolling statistics that encode autoregressive demand behavior
- **High-accuracy XGBoost Model:** delivering ~94% forecast accuracy with interpretable feature importance outputs
- **Production Dashboard:** an interactive Streamlit application enabling real-time, meal-level demand exploration for business teams

☒ **Business Impact**

By forecasting demand with ~94% accuracy, businesses can reduce food waste by an estimated 20–35%, eliminate stockouts during peak periods, and optimize procurement cycles — translating to measurable cost savings and revenue protection for food service operators of all scales.

References

- XGBoost Documentation — Chen & Guestrin (2016), [xgboost.readthedocs.io](https://xgboost.ai)
- Streamlit Framework — streamlit.io/docs
- Genpact Food Demand Forecasting Dataset — Kaggle Competition
- Pandas Time Series Documentation — pandas.pydata.org
- Scikit-Learn Metrics — scikit-learn.org/stable/modules/model_evaluation